

40+ DSA Patterns That Cover 99% of Interview Questions

Stop solving random problems. Learn these patterns and you'll recognize any interview question in seconds.

How to Use This Guide

1. Learn one pattern at a time (2-3 days per pattern)
2. Solve the must-do problems for each pattern
3. Before solving – identify which pattern fits and WHY
4. Move to the next pattern only when you can solve a new problem using the current one
5. Revise patterns, not individual problems

Pattern 1: Two Pointers

When to use: Sorted arrays, finding pairs, removing duplicates, comparing from both ends

How it works: Two pointers moving towards each other or in the same direction

#	Problem	Difficulty	Company Frequency
1	Two Sum II (sorted array)	Easy	Very High
2	3Sum	Medium	Very High
3	Container With Most Water	Medium	Very High
4	Remove Duplicates from Sorted Array	Easy	High
5	Trapping Rain Water	Hard	Very High

Recognition triggers:

Sorted array + find pair/triplet

"In-place" operation on array

Compare elements from both ends

Palindrome checking

Pattern 2: Sliding Window

When to use: Subarray/substring problems, contiguous sequence, maximum/minimum of fixed or variable window

How it works: Expand window from right, shrink from left when condition breaks

#	Problem	Difficulty	Company Frequency
1	Maximum Sum Subarray of Size K	Easy	High
2	Longest Substring Without Repeating Characters	Medium	Very High
3	Minimum Window Substring	Hard	Very High
4	Longest Repeating Character Replacement	Medium	High
5	Permutation in String	Medium	High

Recognition triggers:

"Contiguous subarray/substring"

"Maximum/minimum of size K"

"Longest/shortest" with a condition

Uses a HashMap + two pointers

Pattern 3: Binary Search

When to use: Sorted arrays, search space reduction, finding boundaries, optimization problems

How it works: Eliminate half the search space each step – $O(\log n)$

#	Problem	Difficulty	Company Frequency
1	Binary Search (basic)	Easy	Very High
2	Find First and Last Position	Medium	Very High
3	Search in Rotated Sorted Array	Medium	Very High
4	Find Peak Element	Medium	High
5	Koko Eating Bananas	Medium	High

Recognition triggers:

Sorted array

"Find minimum/maximum that satisfies condition"

"Search in rotated/modified sorted"

$O(\log n)$ requirement

"Minimize the maximum" or "maximize the minimum"

Pattern 4: Fast & Slow Pointers (Floyd's)

When to use: Cycle detection, finding middle, finding nth from end in linked lists

How it works: Two pointers moving at different speeds – they meet if cycle exists

#	Problem	Difficulty	Company Frequency
1	Linked List Cycle	Easy	Very High
2	Linked List Cycle II (find start)	Medium	High
3	Middle of Linked List	Easy	High
4	Happy Number	Easy	Medium
5	Find the Duplicate Number	Medium	High

Recognition triggers:

"Detect cycle"

"Find middle"

"Find nth from end"

Linked list problems

Sequence that eventually repeats

Pattern 5: Merge Intervals

When to use: Overlapping intervals, scheduling, merging ranges

How it works: Sort by start time, merge if current overlaps with previous

#	Problem	Difficulty	Company Frequency
1	Merge Intervals	Medium	Very High
2	Insert Interval	Medium	High

3	Non-overlapping Intervals	Medium	High
---	---------------------------	--------	------

#	Problem	Difficulty	Company Frequency
4	Meeting Rooms	Easy	High
5	Meeting Rooms II	Medium	Very High

Recognition triggers:

"Intervals" or "ranges"
 "Overlapping"
 "Schedule" or "meetings"
 Start time + end time given

Pattern 6: BFS (Breadth-First Search)

When to use: Shortest path (unweighted), level-order traversal, finding nearest, spreading/infection problems

How it works: Queue-based, process all nodes at current level before moving to next

#	Problem	Difficulty	Company Frequency
1	Binary Tree Level Order Traversal	Medium	Very High
2	Rotting Oranges	Medium	Very High
3	Number of Islands (BFS version)	Medium	Very High
4	Word Ladder	Hard	High
5	Shortest Path in Binary Matrix	Medium	High

Recognition triggers:

"Shortest path" (unweighted graph)
 "Level by level"
 "Nearest" or "minimum steps"
 "Spreading" (fire, infection, water)
 Matrix traversal with minimum moves

Pattern 7: DFS (Depth-First Search)

When to use: Path finding, connectivity, exhaustive search, tree traversals, backtracking base

How it works: Stack-based (or recursion), go as deep as possible before backtracking

#	Problem	Difficulty	Company Frequency
1	Number of Islands	Medium	Very High
2	Max Area of Island	Medium	High
3	Clone Graph	Medium	High
4	Path Sum	Easy	High
5	Course Schedule (cycle detection)	Medium	Very High

Recognition triggers:

"Find all paths"

"Connected components"

"Is there a path?"

Tree/graph traversal

"Visit all" or "explore all"

Pattern 8: Backtracking

When to use: All combinations/permutations, constraint satisfaction, sudoku-type problems, generating subsets

How it works: Try a choice → recurse → undo choice (backtrack) → try next

#	Problem	Difficulty	Company Frequency
1	Subsets	Medium	Very High
2	Permutations	Medium	Very High
3	Combination Sum	Medium	Very High
4	N-Queens	Hard	High

5	Word Search	Medium	Very High
---	-------------	--------	-----------

Recognition triggers:

"All possible" combinations/permutations

"Generate all"

Constraint satisfaction

Decision tree with choices at each step

"Place N items following rules"

Pattern 9: Stack

When to use: Matching brackets, next greater/smaller element, expression evaluation, monotonic problems

How it works: LIFO – push when needed, pop when condition met

#	Problem	Difficulty	Company Frequency
1	Valid Parentheses	Easy	Very High
2	Min Stack	Medium	High
3	Daily Temperatures (next greater)	Medium	Very High
4	Largest Rectangle in Histogram	Hard	High
5	Evaluate Reverse Polish Notation	Medium	High

Recognition triggers:

"Matching" or "valid" brackets/parentheses

"Next greater/smaller element"

"Monotonic" increasing/decreasing

"Expression evaluation"

Nested structures

Pattern 10: Heap / Priority Queue

When to use: K-th largest/smallest, top-K elements, merge K sorted, scheduling by priority

How it works: Maintain a min/max heap of size K – $O(n \log k)$ instead of $O(n \log n)$

#	Problem	Difficulty	Company Frequency
1	Kth Largest Element in Array	Medium	Very High
2	Top K Frequent Elements	Medium	Very High
3	Merge K Sorted Lists	Hard	Very High
4	Find Median from Data Stream	Hard	High
5	Task Scheduler	Medium	High

Recognition triggers:

"K-th largest/smallest"

"Top K" anything

"Merge K sorted"

"Median" in a stream

Priority-based scheduling

Pattern 11: HashMap / HashSet

When to use: Frequency counting, finding duplicates, O(1) lookups, two-sum type problems

How it works: Store seen elements for instant lookup

#	Problem	Difficulty	Company Frequency
1	Two Sum	Easy	Every Interview
2	Group Anagrams	Medium	Very High
3	Longest Consecutive Sequence	Medium	Very High
4	Subarray Sum Equals K	Medium	Very High
5	Valid Anagram	Easy	High

Recognition triggers:

"Find pair/group"

"Frequency" or "count"

"Duplicate" detection

Need O(1) lookup

"Complement" finding (target - current)

Pattern 12: Dynamic Programming (1D)

When to use: Optimal substructure + overlapping subproblems, counting ways, min/max cost path

How it works: Break into subproblems, store results, build up from base case

#	Problem	Difficulty	Company Frequency
1	Climbing Stairs	Easy	Very High
2	House Robber	Medium	Very High
3	Coin Change	Medium	Very High

#	Problem	Difficulty	Company Frequency
4	Longest Increasing Subsequence	Medium	Very High
5	Word Break	Medium	High

Recognition triggers:

"How many ways"

"Minimum/maximum cost"

"Can you reach" or "is it possible"

Choices at each step with optimal substructure

Fibonacci-like recurrence

Pattern 13: Dynamic Programming (2D)

When to use: Grid problems, string matching, two sequences comparison

How it works: 2D table where $dp[i][j]$ depends on previous cells

#	Problem	Difficulty	Company Frequency
1	Unique Paths	Medium	Very High
2	Longest Common Subsequence	Medium	Very High
3	Edit Distance	Medium	High

4	0/1 Knapsack	Medium	Very High
5	Longest Palindromic Substring	Medium	Very High

Recognition triggers:

Two strings/sequences comparison

Grid traversal with constraints

"Transform X to Y"

"Longest common" anything

Knapsack-type (pick or skip)

Pattern 14: Greedy

When to use: Local optimal leads to global optimal, interval scheduling, activity selection

How it works: Make the best choice at each step without looking back

#	Problem	Difficulty	Company Frequency
1	Jump Game	Medium	Very High
2	Jump Game II	Medium	High
3	Gas Station	Medium	High
4	Assign Cookies	Easy	Medium
5	Partition Labels	Medium	High

Recognition triggers:

"Minimum number of" steps/operations

Interval scheduling (sort by end time)

"Can you reach"

Locally optimal = globally optimal (prove it)

Activity selection type

Pattern 15: Trees (DFS/BFS)

When to use: Any binary tree problem – traversals, properties, construction

How it works: Recursive DFS (preorder/inorder/postorder) or BFS (level order)

#	Problem	Difficulty	Company Frequency
1	Maximum Depth of Binary Tree	Easy	Very High
2	Validate Binary Search Tree	Medium	Very High
3	Lowest Common Ancestor	Medium	Very High
4	Binary Tree Right Side View	Medium	High
5	Serialize and Deserialize Binary Tree	Hard	High

Recognition triggers:

"Binary tree" in the problem

"Depth", "height", "level"

"Ancestor", "path"

"Validate BST"

Tree construction from traversals

Pattern 16: Graphs (Advanced)

When to use: Topological sort, shortest path (weighted), union-find, connected components

How it works: Depends on sub-pattern (Dijkstra, Kahn's, Union-Find)

#	Problem	Difficulty	Company Frequency
1	Course Schedule (Topological Sort)	Medium	Very High
2	Course Schedule II (order)	Medium	High
3	Network Delay Time (Dijkstra)	Medium	High
4	Number of Connected Components	Medium	High
5	Redundant Connection (Union-Find)	Medium	High

Recognition triggers:

"Prerequisites" or "dependencies" → Topological Sort

"Shortest path" with weights → Dijkstra

"Connected/groups/clusters" → Union-Find

"Cycle in directed graph" → Topological Sort

"Minimum spanning tree" → Kruskal/Prim

Pattern 17: Trie (Prefix Tree)

When to use: Prefix-based search, autocomplete, word dictionary, XOR problems **How it**

works: Tree where each node represents a character, shared prefixes share paths

#	Problem	Difficulty	Company Frequency
1	Implement Trie	Medium	High
2	Word Search II	Hard	High
3	Design Add and Search Words	Medium	High
4	Longest Word in Dictionary	Medium	Medium
5	Replace Words	Medium	Medium

Recognition triggers:

"Prefix" search

"Autocomplete"

"Dictionary" of words

Multiple string searches

"Starts with"

Pattern 18: Bit Manipulation

When to use: Single number problems, power of 2, counting bits, XOR tricks

How it works: AND, OR, XOR, shift operations on binary representation

#	Problem	Difficulty	Company Frequency
1	Single Number	Easy	Very High
2	Number of 1 Bits	Easy	High
3	Counting Bits	Easy	Medium

4	Reverse Bits	Easy	Medium
5	Missing Number	Easy	High

Recognition triggers:

"Single/unique number" in array of duplicates

"Power of 2"

"Binary representation"

"XOR" mentioned

"Without extra space" + duplicate finding

Pattern 19: Monotonic Stack/Queue

When to use: Next greater/smaller element, sliding window maximum, stock span problems

How it works: Maintain stack/queue in increasing or decreasing order

#	Problem	Difficulty	Company Frequency
1	Next Greater Element I	Easy	High
2	Daily Temperatures	Medium	Very High
3	Sliding Window Maximum	Hard	High

#	Problem	Difficulty	Company Frequency
4	Stock Span Problem	Medium	High
5	Largest Rectangle in Histogram	Hard	High

Recognition triggers:

"Next greater/smaller"

"Sliding window maximum/minimum"

"Span" problems

"For each element, find the nearest..."

Needs $O(n)$ but brute force is $O(n^2)$

Pattern 20: Union-Find (Disjoint Set)

When to use: Grouping/clustering, detect cycle in undirected graph, connected components, dynamic connectivity

How it works: find() to get root, union() to merge sets – path compression + rank for $O(\alpha(n))$

#	Problem	Difficulty	Company Frequency
1	Number of Connected Components	Medium	High
2	Redundant Connection	Medium	High
3	Accounts Merge	Medium	High
4	Number of Islands II (online)	Hard	Medium
5	Longest Consecutive Sequence (UF approach)	Medium	Medium

Recognition triggers:

"Groups" or "clusters" forming over time

"Are X and Y connected?"

"Merge" operations

Undirected graph cycle detection

Dynamic connectivity (edges added over time)

Pattern 21: Binary Search on Answer

When to use: Optimization problems where you need to find minimum/maximum value that satisfies a condition

How it works: Binary search on the answer space (not the array) – check if mid is feasible, adjust accordingly

#	Problem	Difficulty	Company Frequency
1	Koko Eating Bananas	Medium	Very High
2	Split Array Largest Sum	Hard	High
3	Capacity To Ship Packages Within D Days	Medium	High
4	Aggressive Cows (Minimum Distance)	Medium	Very High

5	Book Allocation Problem	Medium	Very High
---	-------------------------	--------	-----------

Recognition triggers:

"Minimize the maximum" or "Maximize the minimum"

"Find minimum X such that condition is satisfied"

Answer lies in a range [low, high] and you can check feasibility

"Allocate/distribute to minimize/maximize"

Monotonic feasibility – if X works, X+1 also works (or vice versa)

Pattern 22: Prefix Sum

When to use: Range sum queries, subarray sum problems, cumulative frequency **How it works:**

Precompute cumulative sums so any range sum = $\text{prefix}[\text{right}] - \text{prefix}[\text{left}-1]$ in $O(1)$

#	Problem	Difficulty	Company Frequency
1	Subarray Sum Equals K	Medium	Very High
2	Range Sum Query (Immutable)	Easy	High
3	Contiguous Array (equal 0s and 1s)	Medium	High
4	Product of Array Except Self	Medium	Very High
5	Find Pivot Index	Easy	Medium

Recognition triggers:

"Sum of subarray" or "range sum"

"Count subarrays with sum = K"

"Equal number of X and Y"

"Product except self"

Repeated range queries on same array

Pattern 23: Kadane's Algorithm

When to use: Maximum subarray sum, maximum product subarray, circular subarray problems **How**

it works: Track current maximum ending here – either extend or restart from current element

#	Problem	Difficulty	Company Frequency
1	Maximum Subarray	Medium	Every Interview
2	Maximum Product Subarray	Medium	Very High
3	Maximum Circular Subarray	Medium	High
4	Maximum Sum Circular Subarray	Medium	High
5	Best Time to Buy and Sell Stock	Easy	Very High

Recognition triggers:

"Maximum/minimum subarray sum"

"Contiguous subarray with maximum"

"Buy and sell stock" (profit = max difference)

"Circular" array max sum

Any problem where you decide "extend current or start new"

Pattern 24: Linked List Patterns

When to use: Reversal, merge, partition, copy, reorder linked list problems

How it works: Pointer manipulation – prev, curr, next tracking

#	Problem	Difficulty	Company Frequency
1	Reverse Linked List	Easy	Every Interview
2	Merge Two Sorted Lists	Easy	Very High
3	Reverse Nodes in K-Group	Hard	Very High
4	Reorder List	Medium	High

#	Problem	Difficulty	Company Frequency
5	Copy List with Random Pointer	Medium	High

Recognition triggers:

- "Reverse" a linked list or portion
- "Merge" sorted lists
- "Reorder" or "rearrange" nodes
- "K-group" operations
- In-place modification (no extra space)

Pattern 25: Matrix / Grid Traversal

When to use: 2D grid problems, spiral order, rotation, island problems, path finding in grid

How it works: Direction arrays (dx, dy), visited tracking, BFS/DFS on grid

#	Problem	Difficulty	Company Frequency
1	Spiral Matrix	Medium	Very High
2	Rotate Image (90 degrees)	Medium	Very High
3	Set Matrix Zeroes	Medium	High
4	Search a 2D Matrix	Medium	High
5	Surrounded Regions	Medium	High

Recognition triggers:

- "2D grid/matrix"
- "Spiral order"
- "Rotate" matrix
- "Flood fill" type
- "Path in grid" with obstacles

Pattern 26: String Patterns

When to use: Pattern matching, anagram, palindrome, parsing, substring problems

How it works: Frequency maps, two pointers on strings, expand around center

#	Problem	Difficulty	Company Frequency
---	---------	------------	-------------------

1	Longest Palindromic Substring	Medium	Very High
2	Valid Palindrome	Easy	High
3	Group Anagrams	Medium	Very High
4	String to Integer (atoi)	Medium	High
5	Longest Common Prefix	Easy	High

Recognition triggers:

"Palindrome" → expand around center or DP

"Anagram" → frequency map comparison

"Pattern matching" → KMP or Rabin-Karp

"Parse" or "convert" string

"Common prefix/suffix"

Pattern 27: Recursion & Divide and Conquer

When to use: Problems that break into identical smaller subproblems, merge sort, quick sort, power calculation

How it works: Solve for smaller input, combine results – $T(n) = 2T(n/2) + O(n)$

#	Problem	Difficulty	Company Frequency
1	Merge Sort	Medium	Very High
2	Quick Sort / Quick Select	Medium	Very High
3	Pow(x, n)	Medium	High
4	Count Inversions	Medium	High
5	Median of Two Sorted Arrays	Hard	High

Recognition triggers:

Problem can be split into two equal halves

"Sort" related

"Count inversions" or "count pairs"

"Kth element" → Quick Select

Merge step combines two sorted results

Pattern 28: Topological Sort

When to use: Dependency ordering, course scheduling, build order, task scheduling with prerequisites

How it works: Kahn's algorithm (BFS with in-degree) or DFS with visited states

#	Problem	Difficulty	Company Frequency
1	Course Schedule	Medium	Very High
2	Course Schedule II (order)	Medium	Very High
3	Alien Dictionary	Hard	High
4	Minimum Height Trees	Medium	Medium
5	Longest Path in DAG	Medium	High

Recognition triggers:

"Prerequisites" or "dependencies"

"Order of tasks"

"Build order"

"Is it possible to finish all?"

Directed Acyclic Graph (DAG)

Pattern 29: Segment Tree / Binary Indexed Tree (BIT)

When to use: Range queries with updates – range sum, range min/max with point updates **How it**

works: Tree structure that stores aggregate info for segments – $O(\log n)$ query and update

#	Problem	Difficulty	Company Frequency
1	Range Sum Query (Mutable)	Medium	High
2	Count of Smaller Numbers After Self	Hard	High
3	Range Minimum Query	Medium	Medium
4	My Calendar (interval booking)	Medium	Medium
5	Count of Range Sum	Hard	Medium

Recognition triggers:

"Range query" WITH "updates" (prefix sum won't work)
"Count of elements in range" that changes
"Mutable" array with range queries
Online queries (can't precompute)
Interval booking/scheduling with conflicts

Pattern 30: Math & Number Theory

When to use: GCD/LCM, prime numbers, modular arithmetic, combinatorics, digit manipulation

How it works: Mathematical formulas and properties — Euclidean algorithm, Sieve, modular exponentiation

#	Problem	Difficulty	Company Frequency
1	Count Primes (Sieve)	Medium	High
2	GCD / LCM	Easy	High
3	Power of Two/Three	Easy	Medium
4	Reverse Integer	Medium	High
5	Factorial Trailing Zeroes	Medium	Medium

Recognition triggers:

"Prime" numbers
"GCD" or "divisors"
"Modulo $10^9 + 7$ "
"Digit" manipulation
"Factorial", "combinatorics", " nCr "
"Power" or "exponentiation"

Pattern 31: Dutch National Flag / 3-Way Partition

When to use: Sort array with 3 categories (0/1/2), partition around pivot, separate colors/types

How it works: Three pointers — low, mid, high. Process mid, swap to correct region.

#	Problem	Difficulty	Company Frequency
1	Sort Colors (0, 1, 2)	Medium	Very High
2	Move Zeroes	Easy	Very High

#	Problem	Difficulty	Company Frequency
3	Segregate Even and Odd	Easy	Medium
4	3-Way Partition	Medium	High
5	Sort Array by Parity	Easy	Medium

Recognition triggers:

"Sort with only 3 distinct values"

"Move zeroes/negatives to front/end"

"Partition into 3 groups"

"In-place" separation of elements by category

"One pass" sorting with limited distinct values

Pattern 32: Cycle Detection in Graphs (Directed & Undirected)

When to use: Detecting loops in dependencies, validating DAGs, detecting infinite loops

How it works: DFS with 3 states (unvisited, in-progress, done) for directed; Union-Find or DFS for undirected

#	Problem	Difficulty	Company Frequency
1	Course Schedule (has cycle?)	Medium	Very High
2	Detect Cycle in Undirected Graph	Medium	Very High
3	Find Eventual Safe States	Medium	Medium
4	Redundant Connection	Medium	High
5	Circular Dependency Detection	Medium	High

Recognition triggers:

"Is it possible to finish all courses/tasks?"

"Detect cycle" / "circular dependency"

"Safe states" (no path to cycle)

Directed graph + "valid ordering exists?"

"Deadlock detection"

Pattern 33: Shortest Path Algorithms

When to use: Find minimum distance/cost between nodes in weighted/unweighted graphs

How it works: Dijkstra (non-negative weights), Bellman-Ford (negative weights), Floyd-Warshall (all pairs)

#	Problem	Difficulty	Company Frequency
1	Network Delay Time (Dijkstra)	Medium	High
2	Cheapest Flights Within K Stops (Bellman-Ford)	Medium	High
3	Path With Minimum Effort	Medium	High
4	Shortest Path in Binary Matrix (0-1 BFS)	Medium	High
5	Find the City With Smallest Number of Neighbors (Floyd)	Medium	Medium

Recognition triggers:

"Shortest/cheapest path" with weights

"Minimum cost to reach"

"At most K steps/stops"

Weighted graph

"All pairs shortest path" → Floyd-Warshall

Pattern 34: Interval DP

When to use: Problems on subarrays/subsequences where you try all possible split points

How it works: $dp[i][j]$ = best answer for subarray $i..j$, try all split points k between i and j

#	Problem	Difficulty	Company Frequency
1	Matrix Chain Multiplication	Medium	Very High

2	Burst Balloons	Hard	High
3	Palindrome Partitioning II (min cuts)	Hard	High
4	Minimum Cost to Merge Stones	Hard	Medium
5	Optimal BST	Medium	Medium

Recognition triggers:

- "Minimum cost to merge/split"
- "Partition into groups optimally"
- "Matrix chain" type (order of operations)
- "All possible splits of a range"
- Subproblem is a contiguous range $[i, j]$

Pattern 35: DP on Subsequences

When to use: Subset sum, knapsack variants, counting subsequences, target sum problems

How it works: For each element decide include or exclude – $dp[i][sum]$ = ways to form sum using first i elements

#	Problem	Difficulty	Company Frequency
1	Subset Sum	Medium	Very High
2	Partition Equal Subset Sum	Medium	Very High
3	Target Sum (+ and - signs)	Medium	High
4	Count of Subsets with Given Sum	Medium	High
5	Minimum Subset Sum Difference	Medium	High

Recognition triggers:

- "Subset/subsequence with sum = target"
- "Partition into two groups"
- "Count ways to reach target"
- "Include or exclude" each element
- Knapsack-type: weight + value with capacity

Pattern 36: DP on Strings

When to use: Edit distance, longest common subsequence/substring, string matching, palindrome problems

How it works: $dp[i][j]$ comparing first i chars of string A with first j chars of string B

#	Problem	Difficulty	Company Frequency
1	Longest Common Subsequence	Medium	Very High

#	Problem	Difficulty	Company Frequency
2	Edit Distance	Medium	Very High
3	Longest Palindromic Subsequence	Medium	High
4	Shortest Common Supersequence	Hard	High
5	Distinct Subsequences	Hard	Medium

Recognition triggers:

"Two strings" + "common" or "transform"
"Edit distance" / "minimum operations"
"Longest palindromic subsequence"
"How many distinct subsequences match"
Compare character by character decision

Pattern 37: DP on Stocks

When to use: Buy/sell stock problems with various constraints (cooldown, fee, k transactions)

How it works: State machine DP – states: holding, not-holding, cooldown

#	Problem	Difficulty	Company Frequency
1	Best Time to Buy and Sell Stock (one transaction)	Easy	Very High
2	Best Time to Buy and Sell Stock II (unlimited)	Medium	High

3	Best Time to Buy and Sell Stock with Cooldown	Medium	High
4	Best Time to Buy and Sell Stock with Transaction Fee	Medium	High
5	Best Time to Buy and Sell Stock III (at most 2)	Hard	High

Recognition triggers:

"Buy and sell stock"

"Maximum profit"

"At most K transactions"

"Cooldown period"

"Transaction fee"

Pattern 38: Sliding Window + HashMap

When to use: Substring problems with character frequency constraints, anagram finding, at most K distinct

How it works: Expand window, track frequencies in map, shrink when constraint violated

#	Problem	Difficulty	Company Frequency
1	Longest Substring with At Most K Distinct Characters	Medium	Very High
2	Find All Anagrams in String	Medium	Very High
3	Minimum Window Substring	Hard	Very High
4	Subarrays with K Different Integers	Hard	High
5	Longest Substring with At Least K Repeating Characters	Medium	Medium

Recognition triggers:

"At most K distinct"

"Find all anagrams"

"Smallest window containing all characters"

"Frequency constraint" on substring

Sliding window but need to track character counts

Pattern 39: Multi-source BFS

When to use: Multiple starting points spreading simultaneously, nearest distance from any source

How it works: Add ALL sources to queue initially, BFS expands from all simultaneously

#	Problem	Difficulty	Company Frequency
1	Rotting Oranges	Medium	Very High
2	01 Matrix (nearest 0 for each cell)	Medium	High
3	Walls and Gates	Medium	High
4	Pacific Atlantic Water Flow	Medium	High
5	Shortest Bridge	Medium	Medium

Recognition triggers:

"Spreading from multiple sources simultaneously"

"Nearest distance from any X"

"Multi-source" / "from all starting points"

"Infection/fire spreading"

Need distance from nearest source for every cell

Pattern 40: Two Heaps

When to use: Finding median in stream, balance two halves, sliding window median

How it works: Max-heap for lower half + Min-heap for upper half, keep sizes balanced

#	Problem	Difficulty	Company Frequency
1	Find Median from Data Stream	Hard	Very High
2	Sliding Window Median	Hard	High
3	IPO (maximize capital)	Hard	Medium
4	Balance a Dataset	Medium	Medium
5	Maximize Score After Applying K Operations	Medium	Medium

Recognition triggers:

"Median" in a stream or window

"Balance two halves"

Need quick access to both max of lower and min of upper

"Running median"

Maintain sorted halves dynamically

Pattern 41: Subsets / Power Set Generation

When to use: Generate all subsets, combinations of size K, power set, bit masking for subset enumeration

How it works: Iterative (add each element to all existing subsets) or bitmask (2^n iterations)

#	Problem	Difficulty	Company Frequency
1	Subsets	Medium	Very High
2	Subsets II (with duplicates)	Medium	High
3	Combinations (size K)	Medium	High
4	Letter Combinations of Phone Number	Medium	Very High

#	Problem	Difficulty	Company Frequency
5	Generate Parentheses	Medium	Very High

Recognition triggers:

"Generate all subsets"

"All combinations of size K"

"Power set"

"Generate all valid" (parentheses, IP addresses)

Can use bitmask for small N ($N \leq 20$)

Pattern 42: Line Sweep

When to use: Event-based interval problems, overlapping intervals count, skyline problem

How it works: Convert intervals to events (start/end), sort events, sweep left to right

#	Problem	Difficulty	Company Frequency
1	Meeting Rooms II (max overlap)	Medium	Very High
2	The Skyline Problem	Hard	High
3	My Calendar II (double booking)	Medium	Medium
4	Car Pooling	Medium	High
5	Points That Intersect With Cars	Easy	Medium

Recognition triggers:

"Maximum number of overlapping intervals"

"Skyline" or "outline"

"At any point in time, how many..."

Event start/end with counting

"Capacity" at different time points

Pattern Recognition Cheat Sheet

Problem Says...	Think This Pattern
"Sorted array + find pair"	Two Pointers

Problem Says...	Think This Pattern
"Contiguous subarray/substring"	Sliding Window
"Sorted + $O(\log n)$ "	Binary Search
"Sorted array + find pair"	Two Pointers
"Contiguous subarray/substring"	Sliding Window
"Sorted + $O(\log n)$ "	Binary Search
"Minimize the maximum / maximize the minimum"	Binary Search on Answer
"Linked list + cycle / middle"	Fast & Slow
"Reverse / merge linked list"	Linked List Patterns
"Intervals/ranges/schedule"	Merge Intervals

"Shortest path (unweighted)"	BFS
"Multiple starting points spreading"	Multi-source BFS
"All paths / explore all"	DFS
"All combinations/permutations"	Backtracking
"Generate all subsets"	Subsets / Power Set
"Next greater/smaller"	Monotonic Stack
"Sliding window max"	Monotonic Queue
"Top K / K-th element"	Heap
"Running median"	Two Heaps
"Find pair / frequency"	HashMap
"Substring with K distinct"	Sliding Window + HashMap
"Range sum / subarray sum = K"	Prefix Sum
"Maximum subarray sum"	Kadane's Algorithm
"How many ways / min cost"	DP (1D)
"Two strings comparison / edit"	DP on Strings
"Subset sum / knapsack"	DP on Subsequences
"Buy and sell stock"	DP on Stocks
"Optimal split of range [i,j]"	Interval DP
"Locally optimal = global"	Greedy

Problem Says...	Think This Pattern
"Binary tree anything"	Tree DFS/BFS
"Prerequisites / order"	Topological Sort
"Detect cycle in graph"	Cycle Detection
"Shortest path with weights"	Dijkstra / Bellman-Ford
"Prefix search / autocomplete"	Trie
"Single number / XOR trick"	Bit Manipulation
"Groups / clusters / connected"	Union-Find

"Sort with 3 values / partition"	Dutch National Flag
"2D grid / spiral / rotate"	Matrix Traversal
"Palindrome / anagram / pattern"	String Patterns
"Split into halves / merge sort"	Divide & Conquer
"Range query + updates"	Segment Tree / BIT
"Maximum overlapping intervals"	Line Sweep
"GCD / prime / modulo"	Math & Number Theory

Study Plan (10 Weeks)

Week	Patterns	Problems/Day
Week 1	Two Pointers, Sliding Window, Binary Search, Binary Search on Answer	3-4
Week 2	HashMap, Prefix Sum, Kadane's, Sliding Window + HashMap	3-4
Week 3	Stack (Monotonic), Linked List Patterns, Fast & Slow, Dutch National Flag	3-4
Week 4	Merge Intervals, Line Sweep, String Patterns, Matrix Traversal	3-4
Week 5	BFS, Multi-source BFS, DFS, Backtracking, Subsets/Power Set	3-4
Week 6	Trees, Topological Sort, Cycle Detection, Shortest Path Algos	3-4
Week 7	DP (1D), DP on Subsequences, DP on Stocks, Greedy	3-4
Week 8	DP on Strings, Interval DP, DP (2D)	3-4
Week 9	Heap, Two Heaps, Trie, Union-Find, Divide & Conquer	3-4
Week 10	Bit Manipulation, Segment Tree/BIT, Math, Revision + Mock Interviews	3-4

Total: ~200 problems in 10 weeks (covers 99% of interviews)

Tips

1. **Pattern first, problem second.** Before coding, identify which pattern applies and WHY.
2. **Don't memorize solutions.** Understand the pattern so you can apply it to unseen problems.
3. **Time yourself.** Medium = 30 min, Hard = 45 min. If stuck after 20 min, look at the hint (not full solution).
4. **Revise weekly.** Solve 1 random problem from each covered pattern every weekend.
5. **Explain**

out loud. If you can't explain your approach in 2 sentences, you don't truly understand it. 6. **Track your weak patterns.** If you fail 2+ problems in a pattern, spend an extra day on it.

*Beyond the Degree – 200 focused problems with patterns > 500 random problems without understanding.
Quality beats quantity every single time.*

Feel free to DM on insta @kaash.sde if any doubt.

@kaash.sde